

NORMAL OPERATION :

The Kohonen layer :

The Kohonen layer functions in a 'winner- take -all fashion'; that is, for given input vector , one and only one Kohonen neuron outputs a logical one ; all other outputs are zero .Associated with each Kohonen neuron it to each input .Kohonen neuron K1 has weights $w_{11}, w_{21}, \dots, w_{m1}$, comprising a weight vector W_1 . These connect by way of the input layer to input signals $x_1, x_2, \dots, x_m$, comprising the input vector X . As with neurons in most networks, the NET output of each Kohonen neuron is simply the summation inputs .This may be expressed as follows :

$$\text{NET } j = w_{1j}x_1 + w_{2j}x_2 + \dots + w_{mj}x_m$$

where NET j is the NET output of kohonen neuron j

$$\text{NET } j = \sum_i x_i w_{ij}$$

or in vector notation

$$N = XW$$

where N is the vector of Kohonen layer NET outputs.

The Kohonen neuron with the largest NET value is the 'winner'. Its output is set to one ; all others are set to zero.

Grossberg Layer:

The Grossberg layer functions in a familiar manner. Its NET output is the weighted sum of the Kohonen layer outputs $k_1, k_2, k_3, \dots, k_n$, forming the vector K . The connecting weight vector designated V consists of the weights $v_{11}, v_{21}, \dots, v_{np}$. The NET output of each Grossberg neuron is then

$$\text{NET } j = \sum_i k_i w_{ij}$$

where NET j is the output of the Grossberg neuron j , or in vector form

$$Y = KV$$

where Y = the Grossberg - layer output vector

K = the Kohonen - layer output vector

V = the Grossberg layer weight matrix

If the Kohonen layer is operated such that one neuron's NET is at one and all others are at zero, only one element of the K vector is nonzero, and the calculation is simple. The only action of each neuron in the Grossberg layer is to output the value of the weight that connects it to the single nonzero Kohonen neuron.

TRAINING THE KOHONEN LAYER :

Kohonen training is a self - organizing algorithm that operates in the supervised mode. For this reason, it is difficult to predict which specific Kohonen neuron will be activated for a given input vector. It is only necessary to ensure that training separates input vectors.

Preprocessing the Input Vectors :

It is highly to normalize all input vector before applying them to the network. This is done by dividing each component of an input vector by that vector's length. This length is found by taking the square root of the sum of the squares of all of the vector's components . In symbols

$$X_i' = X_i / (X_1^2 + X_2^2 + \dots + X_n^2)^{1/2}$$

This converts an input vector into a unit vector pointing in the same direction ; that is , a vector of unit length in n-dimensional space.

To train the Kohonen layer , an input vector is applied and its dot product is calculated with the weight vector associated with each Kohonen neuron. The neuron with the highest dot product is declared the "winner" and its weights are adjusted . Because the dot product operation used to calculate the NET values is a measure of similarity between the input and weight vectors the training process actually consists of selecting the Kohonen neuron whose weight is most similar to the input vector, and it still more similar . The network self - organizes so that a given Kohonen neuron has maximum output for a given input vector . The training equation that follows is used .

$$W_{new} = W_{old} + (x - W_{old})$$

where

W_{new} = the new value of a weight connecting an input component x to the winning neuron

W_{old} = the previous value of this weight

α = a training rate coefficient that may vary during the training process

In the interpolative mode, a group of the Kohonen neurons having the highest outputs is allowed to present its outputs to the Grossberg layer. The number of neurons in this group must be chosen for the application, and there is no conclusive evidence regarding an optimum size. Once the group is determined, its set of NET outputs is treated as a vector and normalized to unit length by dividing each NET value by the square root of the sum of the squares of the NET values in the group. Each weight associated with the winning Kohonen neuron is changed by an amount proportional to the difference between its value and the value of the input to which it connects. The direction of the change minimizes the difference between the weight its input. The variable α is a training rate coefficient that usually starts out at 0.7 and may be gradually reduced during training. This allows large initial steps for rapid, coarse training and smaller steps as the final value approached.

All neurons not in the group have their outputs set to zero. If only one input vector were to be associated with each Kohonen neuron, the Kohonen layer could be trained with a single calculation per weight. The weights of a winning neuron would be made equal to the components of the training vector ($\alpha = 1$). Usually the training set includes many input vectors that are similar and the network should be trained to activate the same Kohonen neuron for each of them. In this case, the weights of that neuron should be the average of the input vectors that will activate it. Setting α to a low value will reduce the effect of each training step, making the final value an average of the input vectors to which it was trained. In this way, the weights associated with a neuron will assume a value near the "center" of the input vectors for which that neuron is the "winner".

Y_i = component i of the vector of desired outputs
Initially α is set approximately 0.1 and is gradually reduced as training progresses.

Interpolative Mode :

In the interpolative mode , a group of the Kohonen neurons having the highest outputs is allowed to present its outputs to the Grossberg layer . The number of neurons in this group must be chosen for the application , and there is no conclusive evidence regarding an optimum size . Once the group is determined , its set of NET outputs is treated as a vector and normalized to unit length by dividing each NET value by the squareroot of the sum of the squares of the NET values in the group . All neurons not in the group have their outputs set to zero.

TRAINING THE GROSSBERG LAYER

An input vector is applied , the Kohonen outputs are established , and the grossberg outputs are calculated as in normal operation . Next , each weight is adjusted only if it connects to a Kohonen neuron having a nonzero output . The amount of the weight adjustment is proportional to the difference between the weight and desired output of the Grossberg neuron to which it connects . In symbols

$$V_{ij} = V_{ij \text{ old}} + (Y_j - V_{ij}) K_i$$

where K_i = the output of Kohonen neuron i (only one Kohonen neuron is nonzero)

Y_j = component j of the vector of desired outputs

Initially K_i is set approximately 0.1 and is gradually reduced as training progresses.

The weights of the grossberg layer will converge to the average values of the desired outputs , whereas the weights of the Kohonen layer are trained to the average values of the inputs. Grossberrg training is supervised ; the algorithm has a desired output to which it trains. The unsupervised , self - organising operation of the Kohonen layer produces outputs at indeterminate positions; these mapped to the desired output of the Grossberg layer.

APPLICATION :

In addition to the usual vector - mapping functions , counter propagation is useful in Data Compression . A counter propagation network can be used to compress data prior to transmission , there by reducing the number of bits that must be sent . Suppose an image to be transmitted . It can be divided into subimages S . Each subimage is further subdivided into pixels (picture elements) . Each subimage is then a vector , the elements of which are the pixels of which the subimage is composed. For simplicity , assume that each pixel is either one (light) or zero (dark) . If there are n pixels in a subimage If there are n pixels in a subimage , then n bits will be required to transmit it. If some distortion can be tolerated , substantially fewer bits are actually required to transmit typical images , thereby allowing an image to be transmitted rapidly. This is possible because of the statistical distribution of sub image vectors. Some occur frequently while others occur so seldom that they can be

approximated roughly. The method of vector quantisation finds these shorter bit strings that best represent subimages .

A Counter propagation network can be used to perform vector quantisation. The set of subimage vectors is used as input to train the kohonen layer in the accertive mode in which only a single neuron is allowed to be 1. The Grossberg weights are trained to produce the binary code of the index of the Kohonen neuron that is 1. For example , if Kohonen neuron 7 is 1 (and the others are all 0), the Grossberg layer will be trained to output 00.....000111 (the binary code for 7).It is this shorter bit string is transmitted .

At the receiving end, an identically trained counterpropagation network accepts the binary code and produces the inverse function , an approximation of the original subimage .

This method has been applied both to speech and images , yielding dat compression ratios of 10:1 to 100:1.The quality has been acceptable, however some distortion of the data at the receiving end is inevitable.

SOURCE CODE LISTING